



Control Statements and Program Development

Chapter 03

Algorithms

- Any computing problem can be solved by executing a series of actions in a specific order.
- An algorithm is a procedure for solving a problem in terms of
 - the actions to execute and
 - the order in which these actions execute
- Specifying the order in which statements (actions) execute in a program is called program control.

Pseudocode

- An informal language that helps you develop algorithms without having to worry about the strict details of Python syntax.
- Similar to everyday English.
- Helps you “think out” a program before attempting to write it in a programming language.
- Carefully prepared pseudocode can easily be converted to a corresponding program.
- Pseudocode normally describes only statements representing the actions that occur (e.g., input, output or calculations) after you convert a program from pseudocode to Python and run the program.

Addition-Program Pseudocode

The following pseudocode algorithm prompts the user to enter two integers, inputs them from the user at the keyboard, adds them, then stores and displays their sum:

Prompt the user to enter the first integer

Input the first integer

Prompt the user to enter the second integer

Input the second integer

Add first integer and second integer, store their sum

Display the numbers and their sum

3.4 Control Statements

- Usually, statements in a program execute in the order in which they're written.
 - Called sequential execution.
- Various Python statements enable you to specify that the next statement to execute may be other than the next one in sequence.
 - Called transfer of control and is achieved with Python control statements.

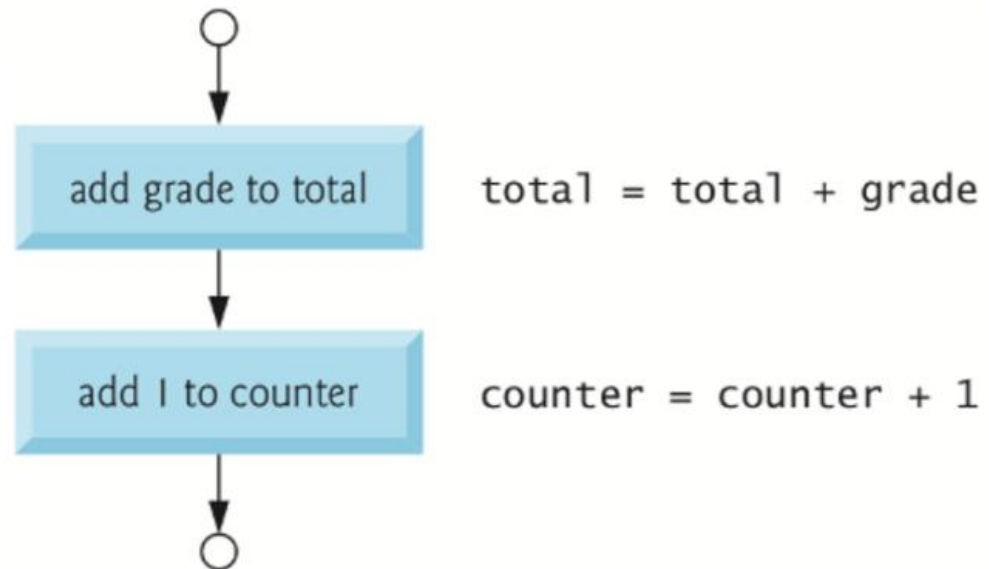
Forms of Control

- All programs can be written using three forms of control
 - Sequential execution
 - The selection statement
 - The iteration statement.
- Python statements execute one after the other "in sequence," unless directed otherwise.

Flowcharts

- Graphical representation of an algorithm or a part of one.
- Drawn using rectangles, diamonds, rounded rectangles and small circles connected by arrows called flowlines.
- Clearly show how forms of control operate.

- Flowchart segment showing sequential execution:



- Rectangle (or action) symbol indicates any action.
- Flowlines show the order in which the actions execute.
- In a flowchart for a complete algorithm, the first symbol is a rounded rectangle containing the word "Begin." The last symbol is a rounded rectangle containing the word "End."
- In a flowchart for only a part of an algorithm, use small circles called connector symbols.

Selection Statements

- Three selection statements that execute code based on a condition—an expression that evaluates to either `True` or `False`:
 - `if` performs an action if a condition is `True` or skips the action if the condition is `False`.
 - `if ... else` statement performs an action if a condition is `True` or performs a different action if the condition is `False`.
 - `if ... elif ... else` statement performs one of many different actions, depending on the truth or falsity of several conditions.
- Anywhere a single action can be placed, a group of actions can be placed.

Iteration Statements

- Two iteration statements
 - `while` repeats an action (or a group of actions) as long as a condition remains `True`.
 - `for` statement repeats an action (or a group of actions) for every item in a sequence of items.

3.5 `if` Statement

- Pseudocode: Suppose that a passing grade on an examination is 60. The pseudocode

```
If student's grade is greater than or equal to 60
    Display 'Passed'
```

- If the condition is true, 'Passed' is displayed. Then, the next pseudocode statement in order is “performed.”
- If the condition is false, nothing is displayed, and the next pseudocode statement is “performed.”
- Indentation emphasizes that 'Passed' is displayed only if the condition is true.

Corresponding `if` Statement

```
In [1]: grade = 85
```

```
In [2]: if grade >= 60:
        print('Passed')
```

Passed

Suite Indentation

- Indenting a suite is required.

```
In [3]: if grade >= 60:  
        print('Passed') # statement is not indented properly
```

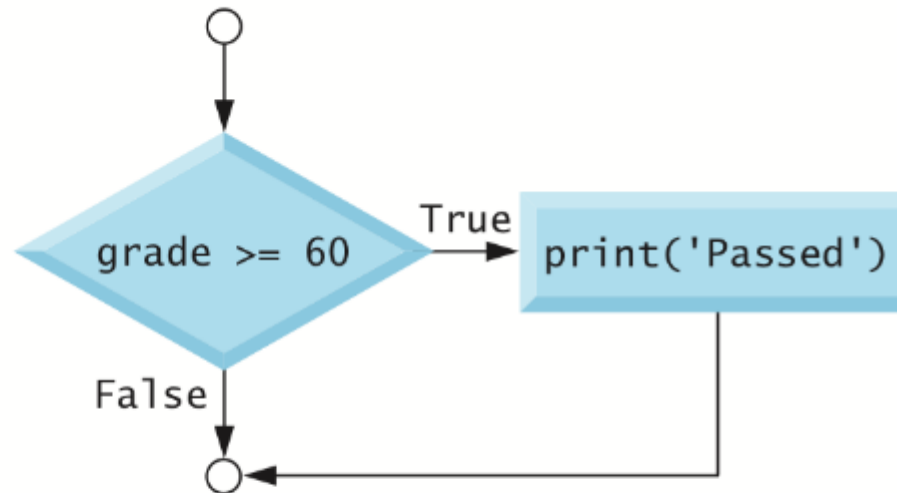
```
File "<ipython-input-3-eb5359d7857b>", line 2  
    print('Passed') # statement is not indented properly  
    ^  
IndentationError: expected an indented block
```

- Statements in a suite must have the same indentation.

```
In [5]: if grade >= 60:  
        print('Passed')  
        print('Good job!')
```

```
File "<tokenize>", line 3  
    print('Good job!')  
    ^  
IndentationError: unindent does not match any outer indentation level
```


if Statement Flowchart



- The decision (diamond) symbol contains a condition that can be either **True** or **False**.
- Two flowlines emerging from it:
 - One indicates the direction to follow when the condition in the symbol is **True**.
 - The other indicates the direction to follow when the condition is **False**.

3.6 `if...else` and `if...elif...else` Statements

- Performs different suites, based on whether a condition is `True` or `False`.
- Pseudocode:

```
If student's grade is greater than or equal to 60  
    Display 'Passed'  
Else  
    Display 'Failed'
```

- Corresponding Python code with variable `grade` initialized to `85`

```
In [1]: grade = 85
```

```
In [2]: if grade >= 60:  
        print('Passed')  
        else:  
        print('Failed')
```

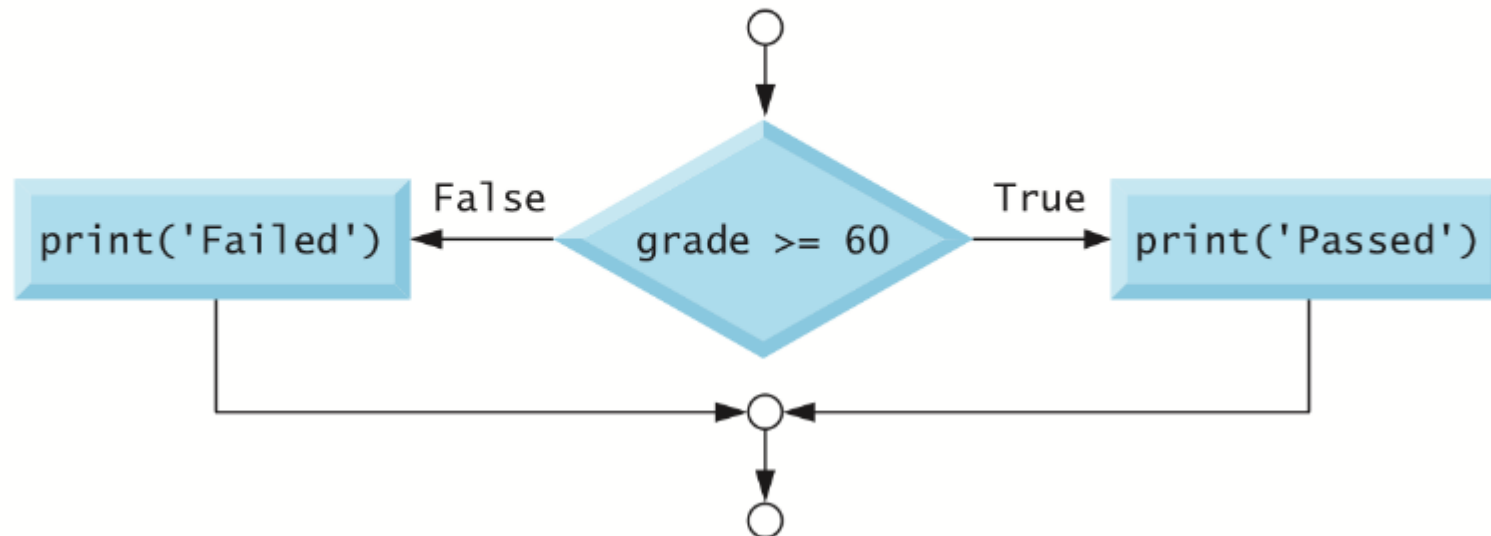
Passed

```
In [3]: grade = 57
```

```
In [4]: if grade >= 60:  
        print('Passed')  
    else:  
        print('Failed')
```

Failed

if ... else Statement Flowchart



Conditional Expressions

- Sometimes the suites in an `if ... else` statement assign different values to a variable, based on a condition

```
In [7]: grade = 87
```

```
In [8]: if grade >= 60:  
        result = 'Passed'  
else:  
        result = 'Failed'
```

```
In [9]: result
```

```
Out[9]: 'Passed'
```

- Can write statements like this using a concise conditional expression.
- The parentheses are not required, but they make it clear that the statement assigns the conditional expression's value to `result`.

```
In [10]: result = ('Passed' if grade >= 60 else 'Failed')
```

```
In [11]: result
```

```
Out[11]: 'Passed'
```

- In interactive mode, you also can evaluate the conditional expression directly.

```
In [12]: 'Passed' if grade >= 60 else 'Failed'
```

```
Out[12]: 'Passed'
```

Multiple Statements in a Suite

In [13]: `grade = 49`

```
In [14]: if grade >= 60:
          print('Passed')
        else:
          print('Failed')
          print('You must take this course again')
```

Failed

You must take this course again

- If you do not indent the second `print`, then it's not in the `else`'s suite.
- In that case the statement always executes, creating strange incorrect output.

In [15]: `grade = 100`

```
In [16]: if grade >= 60:
          print('Passed')
        else:
          print('Failed')
        print('You must take this course again')
```

Passed

You must take this course again

if ... elif ... else Statement

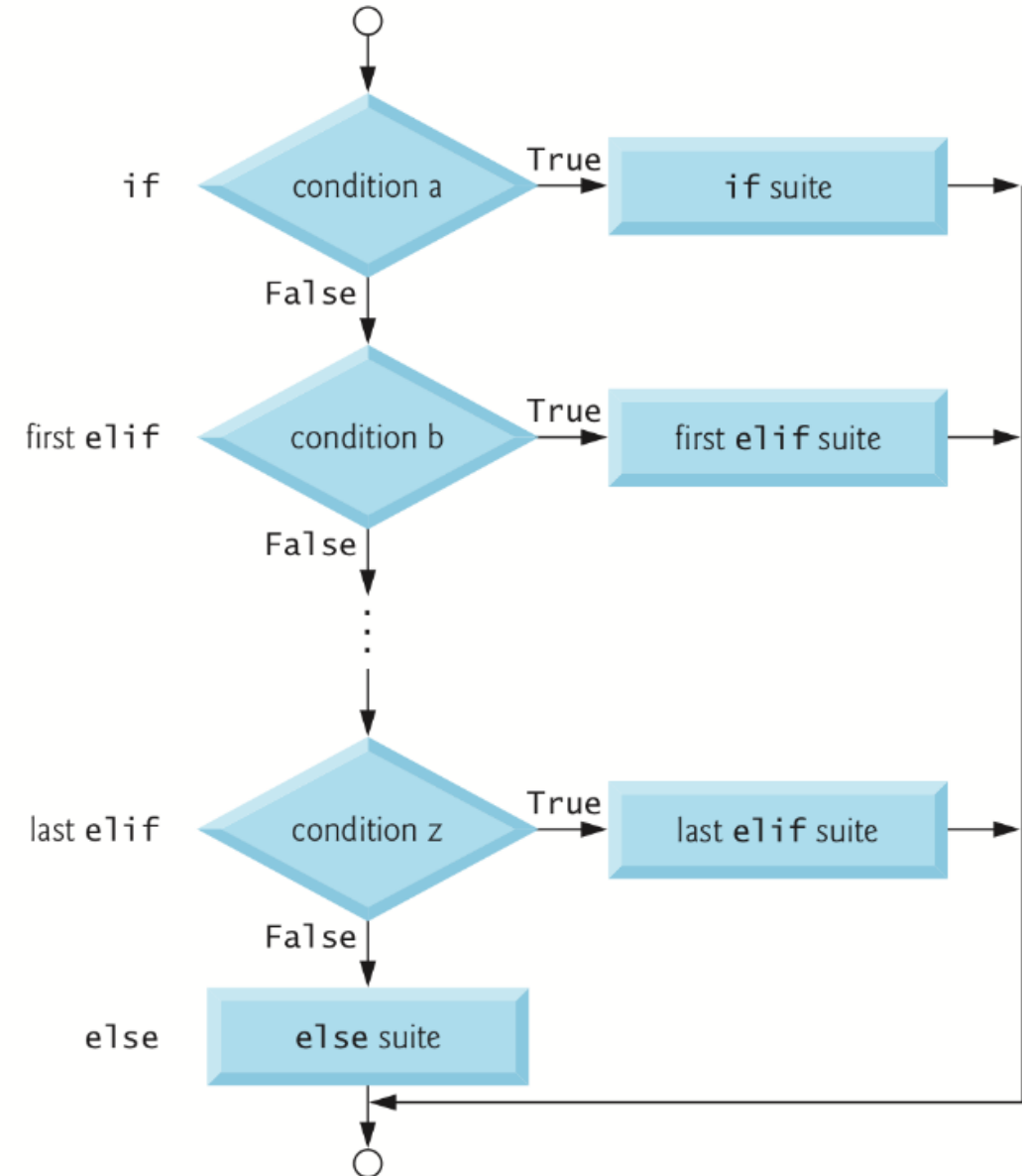
- Can test for many cases.
- Only the action for the first **True** condition executes.

In [17]: `grade = 77`

```
In [18]: if grade >= 90:
          print('A')
        elif grade >= 80:
          print('B')
        elif grade >= 70:
          print('C')
        elif grade >= 60:
          print('D')
        else:
          print('F')
```

C

if ... elif ... else Statement Flowchart



else Is Optional

- Handle values that do not satisfy any of the conditions.
- Without an `else`, if no conditions are `True`, the program does not execute any of the statement's suites.

3.7 while Statement

- Repeats one or more actions while a condition remains `True`.

```
In [1]: product = 3
```

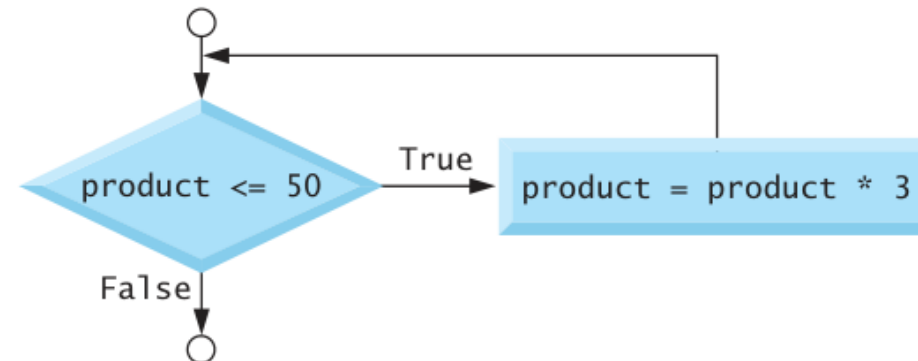
```
In [2]: while product <= 50:  
        product = product * 3
```

```
In [3]: product
```

```
Out[3]: 81
```

- To prevent an infinite loop, something in the `while` suite must change `product`'s value, so the condition eventually becomes `False`.

while Statement Flowchart



3.8 for Statement

- Repeat an action or several actions for each item in a sequence of items.
- A string is a sequence of individual characters.

```
In [1]: for character in 'Programming':  
        print(character, end=' ')
```

P r o g r a m m i n g

- Upon entering the `for` loop, Python assigns the 'P' in 'Programming' to the **target variable** between keywords `for` and `in`.
- After executing the suite, Python assigns to `character` the next item in the sequence (that is, the 'r' in 'Programming'), then executes the suite again.
- Continues while there are more items in the sequence.
- Using the target variable in the suite is common but not required.

Function `print`'s `end` Keyword Argument

- `print` displays its argument(s), then moves the cursor to the next line.
- Can change this behavior with the argument `end` :

```
print(character, end='')
```

- `end` is a **keyword argument**, but it's not a Python keyword.
- The *Style Guide for Python Code* recommends placing no spaces around a keyword argument's `=`.
- Keyword arguments are sometimes called named arguments.

Function `print`'s `sep` Keyword Argument

- Keyword argument `sep` (short for separator) specifies the string that appears between the items that `print` displays.
- A space character by default.
- To remove the spaces, use an empty string with no characters between its quotes.

```
In [2]: print(10, 20, 30, sep=',')
```

```
10, 20, 30
```

3.8.1 Iterables, Lists and Iterators

- The sequence to the right of the `for` statement's in keyword must be an iterable.
 - An object from which the `for` statement can take one item at a time.
- One of the most common iterables is a list, which is a comma-separated collection of items enclosed in square brackets (`[` and `]`).

```
In [3]: total = 0
```

```
In [4]: for number in [2, -3, 0, 17, 9]:  
        total = total + number
```

```
In [5]: total
```

```
Out[5]: 25
```

- Each sequence has an iterator.
- The `for` statement uses the iterator “behind the scenes” to get each consecutive item until there are no more to process.

3.8.2 Built-In `range` Function and Generators

- Creates an iterable object that represents a sequence of consecutive integer values starting from 0 and continuing up to, but not including, the argument value.

```
In [6]: for counter in range(10):  
        print(counter, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9
```

Off-By-One Errors

A logic error known as an off-by-one error occurs when you assume that `range`'s argument value is included in the generated sequence.

3.9 Augmented Assignments

- Abbreviate assignment expressions in which the same variable name appears on the left and right of the `=`.

```
In [1]: total = 0
```

```
In [2]: for number in [1, 2, 3, 4, 5]:  
        total += number  # add number to total and store in number
```

```
In [3]: total
```

```
Out[3]: 15
```

- In the following table assume: `c = 3`, `d = 5`, `e = 4`, `f = 2`, `g = 9`, `h = 12`

Augmented assignment	Sample expression	Explanation	Assigns
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to <code>c</code>
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to <code>d</code>
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to <code>e</code>
<code>**=</code>	<code>f **= 3</code>	<code>f = f ** 3</code>	8 to <code>f</code>
<code>/=</code>	<code>g /= 2</code>	<code>g = g / 2</code>	4.5 to <code>g</code>
<code>//=</code>	<code>g //= 2</code>	<code>g = g // 2</code>	4 to <code>g</code>
<code>%=</code>	<code>h %= 9</code>	<code>h = h % 9</code>	3 to <code>h</code>

3.10 Program Development: Sequence-Controlled Iteration

- Most challenging part of solving a problem on a computer is developing an algorithm for the solution.
- Once a correct algorithm has been specified, creating a working Python program from the algorithm is typically straightforward.

3.10.1 Requirements Statement

- A **requirements statement** describes what a program is supposed to do, but not how the program should do it.
- Consider the following simple requirements statement:

A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.

- Once you know the problem's requirements, you can begin creating an algorithm to solve it. Then, you can implement that solution as a program.
- The algorithm for solving this problem must:
 - Keep a running total of the grades.
 - Calculate the average—the total of the grades divided by the number of grades.
 - Display the result.

3.10.2 Pseudocode for the Algorithm

Set total to zero

Set grade counter to zero

Set grades to a list of the ten grades

For each grade in the grades list:

Add the grade to the total

Add one to the grade counter

Set the class average to the total divided by the number of grades

Display the class average

- Note the mentions of **total** and **grade counter**.
- We'll use these in the script to calculate the average.
- Variables for totaling and counting normally are initialized to zero.

3.10.3 Coding the Algorithm in Python

```
# fig03_01.py
"""Class average program with sequence-controlled iteration."""

# initialization phase
total = 0 # sum of grades
grade_counter = 0
grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94] # list of 10 grades

# processing phase
for grade in grades:
    total += grade # add current grade to the running total
    grade_counter += 1 # indicate that one more grade was processed

# termination phase
average = total / grade_counter
print(f'Class average is {average}')
```

In [1]: run fig03_01.py

Class average is 81.7

Execution Phases

- Initialization phase creates the variables needed to process the grades and set these variables to appropriate initial values.
- Processing phase processes the grades, calculating the running total and counting the number of grades processed so far.
- Termination phase calculates and displays the class average.
- Many scripts can be decomposed into these three phases.

3.10.4 Introduction to Formatted Strings

- An **f-string** (short for formatted string) allows inserting values into a string.
- The letter **f** before the string's opening quote indicates it's an f-string.
- You specify where to insert values by using placeholders delimited by curly braces (**{** and **}**).
- `{average}` converts the variable `average`'s value to a string representation, then replaces `{average}` with that `&replacement text`.
- Replacement-text expressions may contain values, variables or other expressions.

3.11 Program Development: Sentinel-Controlled Iteration

- Generalized class-average problem requirements statement:

Develop a class-averaging program that processes an arbitrary number of grades each time the program executes.

- Does not state what the grades are or how many there are.
- The program processes an arbitrary number of grades.
- Use a **sentinel value** (also called a **signal value**, a **dummy value** or a **flag value**) to indicate “end of data entry.”
- Sentinel-controlled iteration is often called **indefinite iteration** because the number of iterations is not known before the loop begins executing.
- A sentinel value must not be confused with any acceptable input value.

Complete Pseudocode Algorithm

Initialize total to zero

Initialize grade counter to zero

Input the first grade (possibly the sentinel)

While the user has not entered the sentinel

Add this grade into the running total

Add one to the grade counter

Input the next grade (possibly the sentinel)

If the counter is not equal to zero

Set the average to the total divided by the counter

Display the average

Else

Display "No grades were entered"

Implementing Sentinel-Controlled Iteration

```
# fig03_02.py
"""Class average program with sentinel-controlled iteration."""

# initialization phase
total = 0 # sum of grades
grade_counter = 0 # number of grades entered

# processing phase
grade = int(input('Enter grade, -1 to end: ')) # get one grade

while grade != -1:
    total += grade
    grade_counter += 1
    grade = int(input('Enter grade, -1 to end: '))

# termination phase
if grade_counter != 0:
    average = total / grade_counter
    print(f'Class average is {average:.2f}')
else:
    print('No grades were entered')
```

```
In [1]: run fig03_02.py
```

```
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 72
Enter grade, -1 to end: -1
Class average is 85.67
```

Formatting the Class Average with Two Decimal Places

- We formatted the class average with two digits to the right of the decimal point.
- In an f-string, you can optionally follow a replacement-text expression with a colon (`:`) and a **format specifier** that describes how to format the replacement text.
- The format specifier `.2f` formats the average as a floating-point number (`f`) with two digits to the right of the decimal point (`.2`).
- Rounds to the hundredths position

3.12 Program Development: Nested Control Statements

Requirements statement:

A college offers a course that prepares students for the state licensing exam for real-estate brokers. Last year, several of the students who completed this course took the licensing examination. The college wants to know how well its students did on the exam. You have been asked to write a program to summarize the results. You have been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam and a 2 if the student failed.

Analyze the results of the exam as follows:

Input each test result (i.e., a 1 or a 2). Display the message "Enter result" each time the program requests another test result.

Count the number of test results of each type.

Display a summary of the test results indicating the number of students who passed and the number of students who failed.

If more than eight students passed the exam, display "Bonus to instructor."

Complete Pseudocode Algorithm

```
Initialize passes to zero
Initialize failures to zero
For each of the ten students
    Input the next exam result
    If the student passed
        Add one to passes
    Else
        Add one to failures
Display the number of passes
Display the number of failures
If more than eight students passed
    Display "Bonus to instructor"
```

Implementing the Algorithm

```
# fig03_03.py
"""Using nested control statements to analyze examination results."""

# initialize variables
passes = 0 # number of passes
failures = 0 # number of failures

# process 10 students
for student in range(10):
    # get one exam result
    result = int(input('Enter result (1=pass, 2=fail): '))

    if result == 1:
        passes = passes + 1
    else:
        failures = failures + 1

# termination phase
print('Passed:', passes)
print('Failed:', failures)

if passes > 8:
    print('Bonus to instructor')
```

In [1]: `run fig03_03.py`

```
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Passed: 6
Failed: 4
```

```
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Passed: 9
Failed: 1
Bonus to instructor
```

3.13 Built-In Function `range`: A Deeper Look

- Function `range`'s two-argument version produces a sequence of consecutive integers from its first argument's value up to, but not including, the second argument's value

```
In [1]: for number in range(5, 10):  
        print(number, end=' ')
```

5 6 7 8 9

- Function `range`'s three-argument version produces a sequence of integers from its first argument's value up to, but not including, the second argument's value, incrementing by the third argument's value (the step)

```
In [2]: for number in range(0, 10, 2):  
        print(number, end=' ')
```

0 2 4 6 8

- If the third argument is negative, the sequence progresses from the first argument's value down to, but not including the second argument's value, decrementing by the third argument's value

```
In [3]: for number in range(10, 0, -2):  
        print(number, end=' ')
```

10 8 6 4 2

Compound-Interest Problem Requirements Statement

Requirements statement:

A person invests \$1000 in a savings account yielding 5% interest. Assuming that the person leaves all interest on deposit in the account, calculate and display the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal),

r is the annual interest rate,

n is the number of years and

a is the amount on deposit at the end of the n^{th} year.

Calculating Compound Interest

- For each year, the loop displays a formatted string containing the year number and the amount on deposit at the end of that year

```
In [15]: for year in range(1, 11):  
         amount = principal * (1 + rate) ** year  
         print(f'{year:>2}{amount:>10.2f}')
```

```
1  1050.00  
2  1102.50  
3  1157.62  
4  1215.51  
5  1276.28  
6  1340.10  
7  1407.10  
8  1477.46  
9  1551.33  
10 1628.89
```

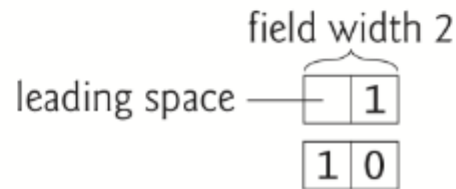
Formatting the Year and Amount on Deposit

```
print(f'{year:>2}{amount:>10.2f}')
```

- Uses an f-string with two placeholders to format the loop's output.

```
{year:>2}
```

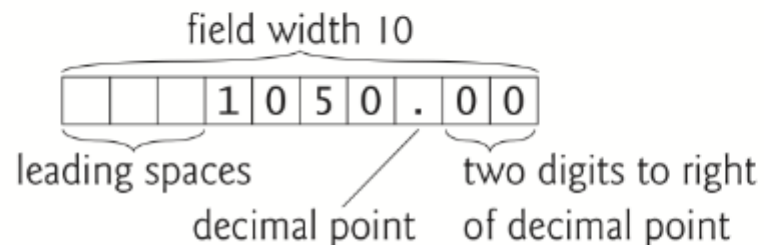
- Uses the format specifier `>2` to indicate that `year`'s value should be **right aligned (>)** in a field of width `2`
- The **field width** specifies the number of character positions to use when displaying the value.



- Can **left align** values with `<`.

```
{amount:>10.2f}
```

- Formats `amount` as a floating-point number (`f`) right aligned (`>`) in a field width of `10` with a decimal point and two digits to the right of the decimal point (`.2`).



3.15 break and continue Statements

- Executing a `break` statement in a `while` or `for` immediately exits that statement.

```
In [1]: for number in range(100):  
        if number == 10:  
            break  
        print(number, end=' ')
```

0 1 2 3 4 5 6 7 8 9

- Executing a `continue` statement in a `while` or `for` loop skips the remainder of the loop's suite.
 - In a `while`, the condition is then tested to determine whether the loop should continue executing.
 - In a `for`, the loop processes the next item in the sequence (if any)

```
In [2]: for number in range(10):  
        if number == 5:  
            continue  
        print(number, end=' ')
```

0 1 2 3 4 6 7 8 9

3.16 Boolean Operators `and`, `or` and `not`

Boolean Operator `and`

- Ensure that two conditions are both `True` with the **Boolean** `and` operator.

```
In [1]: gender = 'Female'
```

```
In [2]: age = 70
```

```
In [3]: if gender == 'Female' and age >= 65:  
        print('Senior female')
```

Senior female

- *Truth table* for the `and` operator:

expression1	expression2	expression1 and expression2
False	False	False
False	True	False
True	False	False
True	True	True

Boolean Operator `or`

- Ensure that one *or* both of two conditions are `True` with the **Boolean `or` operator**.

```
In [4]: semester_average = 83
```

```
In [5]: final_exam = 95
```

```
In [6]: if semester_average >= 90 or final_exam >= 90:  
        print('Student gets an A')
```

Student gets an A

- *Truth table* for the `or` operator:

expression1	expression2	expression1 or expression2
False	False	False
False	True	True
True	False	True
True	True	True

Boolean Operator `not`

- “Reverse” the meaning of a condition.
- **Unary operator**—it has only *one* operand.

```
In [7]: grade = 87
```

```
In [8]: if not grade == -1:  
        print('The next grade is', grade)
```

The next grade is 87

```
In [9]: if grade != -1:  
        print('The next grade is', grade)
```

The next grade is 87

- Truth table for the `not` operator.

expression	not expression
False	True
True	False

3.17 Intro to Data Science: Measures of Central Tendency—Mean, Median and Mode

- **Measures of central tendency:**
 - **mean**—the *average value* in a set of values.
 - **median**—the *middle value* when all the values are arranged in sorted order.
 - **mode**—the *most frequently occurring value*.
- Each represents a “central” value in a set of values.
 - A value which is in some sense typical of the others.

```
In [1]: grades = [85, 93, 45, 89, 85]
```

```
In [2]: sum(grades) / len(grades)
```

```
Out[2]: 79.4
```

- `sum` and `len` are both examples of functional-style programming reductions
- The Python Standard Library’s `statistics` module provides functions for calculating the **reductions** mean, median and mode.

```
In [3]: import statistics
```

```
In [4]: statistics.mean(grades)
```

```
Out[4]: 79.4
```

```
In [5]: statistics.median(grades)
```

```
Out[5]: 85
```

```
In [6]: statistics.mode(grades)
```

```
Out[6]: 85
```

- Sorting `grades` helps you see the median and mode.

```
In [7]: sorted(grades)
```

```
Out[7]: [45, 85, 85, 89, 93]
```

- To sort in reverse order, use:
`sorted(grades, reverse=True)`